

COS125 Homework 2

Project: Campus Dining & Budget Planner

PROJECT FOCUS Build upon your foundational problem solving through coding knowledge use Python to implement sequence structures, loops, sequence operations, indexing, and random variation to simulate and analyze multi-day campus dining spending.

I. General Rules & Academic Integrity

- **Topics:** Practicing foundational programming concepts (using Python) implementing newly introduced course material: while loops, for loops, lists, indexing (lists and strings), creating numerical lists, mixed data types, sequence functions (sum(), len()), and the random module.
- **Concepts Used:** Only use concepts presented in the course to date.
- **Resources:** You may reference the course AI Companion, textbook, lecture slides, and official course tutoring. Record tutor names in your comment header if applicable.
 - **AI Chat Log:** If using the Course AI Companion, save your chat as yourName_aichat.txt and include it in your submission.
- **Human Design:** Non-course AI tools, external searches, and assistance from non-course personnel are strictly prohibited.
- **Grading:** Refer to the provided Homework 2 rubric.

II. Submission Requirements

- **Folder Name:** yourName_diningPlanner
 - **Place all files inside this folder.** Folders should not have .py in their name.
- **File Name:** yourName_hmwk2.py (e.g., alexSmith_hmwk2.py)
- **Planning Document:** yourName_planning (IPO table & pseudocode in .txt, .pdf, or image format).
- **Reflection Document:** yourName_reflection (.txt or .pdf) detailing architectural choices.
 - **AI Chat Log:** If using the Course AI Companion, save your chat as yourName_aichat.txt and include it in your submission.
- **Comment Header:** Mandatory header at the top of your Python file.
- **Submission:** Compress into a .zip archive containing all project files and upload to the portal.

III. Assignment Description In this project, you will build a Python program that simulates a multi-day campus dining allowance spending tracker. The program collects user meal preferences, iterates through menu offerings using a for loop, runs a continuous daily simulation using a while loop until a dining balance runs out, tracks purchases in a list, and uses built-in sequence utilities (sum(), len()) to analyze daily meal costs.

IV. Project Requirements

1. Setup & Documentation

- Create your planning document yourName_planning containing an Input-Processing-Output (IPO) model and pseudocode.

- Create yourName_reflection addressing structural code decisions. Why you chose different coding structures for different part of your code (data types, logic vs sequence vs loops, etc.).
- Include the mandatory comment header in yourName_hmwk2.py.

2. User Input & List Construction

- Prompt the user for:
 - Student name.
 - Student's starting dining card balance in dollars (e.g., 50.00).
- Define a list containing default campus dining halls (e.g., ["Student Union", "North Dining", "Library Cafe"] etc).
- Prompt the user to enter one custom dining spot, then append it to your venue list.

3. Data Processing & Decision Logic

- **Venue Directory (for loop):** Iterate through the dining venue list using a for loop and list indexing to print each location formatted with its numerical index (e.g., Venue 1: Student Union).
- **Simulation Loop (while loop):** Run a continuous simulation that deducts meal costs until the remaining dining balance falls below \$5.00.
 - For each simulated day:
 - Generate a random daily spending total between \$8.00 and \$16.00 using random.random, random.choice, or random.randrange. Adjust the number as needed, based on how the specific random method functions.
 - Append the daily expense to a daily_expenses list.
 - Subtract the daily expense from the running balance.
- **Summary Calculations:**
 - Use sum() to calculate total overall spending across all simulated days.
 - Calculate the average daily expenditure using sum() and len() on your list.
- **Display output**
 - Format the print statements to create a readable output of the information, see example below.

4. Code Reflection Questions (yourName_reflection)

In a separate document, answer the following in 2–3 concise paragraphs:

1. **Loop Selection:** Explain why a while loop is applicable for the spending simulation instead of a for loop, and why a for loop more applicable to output the dining venue directory.
2. **Data Structure Choice:** Detail how storing daily expenses in a Python list simplifies tracking and calculating summary statistics (sum(), average) compared to using individual scalar variables (e.g., day1_cost, day2_cost).

V. Trace Walkthrough & Sample Output

Sample Console Output:

```
=== Campus Dining Allowance Tracker ===
Enter Student Name: Jordan
```

```
Enter Starting Dining Balance ($): 45.00
Enter a custom campus dining spot: South Bear Express
Available Dining Venues:
Venue 1: Student Union
Venue 2: North Dining
Venue 3: Library Cafe
Venue 4: South Bear Express
--- Starting Simulation ---
Day 1: Spent $12.40 | Balance Remaining: $32.60
Day 2: Spent $9.15 | Balance Remaining: $23.45
Day 3: Spent $14.80 | Balance Remaining: $8.65
Day 4: Spent $10.20 | Balance Remaining: -$1.55
Simulation Ended: Low balance threshold reached.
=== Summary Report for Jordan ===
Total Days Simulated: 4
Total Spent: $46.55
Average Daily Expense: $11.64
```

Before You Submit Checklist

- ☐ Planning document included (yourName_planning).
- ☐ Comment header complete and updated in the Python file.
- ☐ Folder (yourName_thermostat) and file (yourName_hmwk1.py) named correctly.
- ☐ AI Companion chat script included, if used.
- ☐ Only permitted concepts used.
- ☐ Program tested with multiple seasonal and budget inputs.
- ☐ All outputs formatted properly using f-strings.

Rubric

Criterion	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations
Problem Interpretation & Logic	Code is concise, clear, and efficient. Uses elegant data representations to model problems and demonstrates a deep understanding of assignment requirements.	Accurately converts problem requirements into functional code. Inputs, data processing, and calculations are complete and accurate.	Attempts to model the problem but contains minor logical or computational errors in processing or output.	Fundamentally misinterprets requirements, omits critical execution steps, or fails to implement core program logic.
Sequence Structures & Indexing	Sequences (lists/strings) are constructed and updated dynamically (e.g., appending). Indexing and sequence functions (i.e. sum(), len()) are applied masterfully without redundancy.	Sequences are correctly defined, updated, and indexed as required. Built-in sequence functions (i.e. sum(), len()) are used correctly for calculations.	Uses sequences, but exhibits minor indexing errors (e.g., off-by-one errors) or misuses list methods and built-in functions.	Fails to use required sequence structures (e.g., relies entirely on scalar variables) or fails to implement indexing correctly.
Iteration & Looping	for and while loops are chosen appropriately for their specific tasks. Loop	Implements required for and while loops correctly.	Implements loops with logical errors (e.g., incorrect	Fails to implement required loops, creates infinite loops, or uses

Criterion	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations
	termination conditions are precise, efficient, and robust against infinite execution.	Loops iterate over sequences properly, terminate as intended, and update state variables accurately.	termination conditions, off-by-one iterations, bad variable updates, or poor loop choice).	incorrect loop control structures.
Conditional Logic & Data Handling	Inputs are stored in descriptive variables with robust data conversion. Conditional logic (if/elif/else) and utility modules (e.g., random) handle edge cases seamlessly.	Prompts for inputs, stores them in descriptive variables, and casts data types correctly. Conditional logic and dynamic functions execute as intended.	Prompts for partial inputs or uses unclear variable names. Minor data type conversion errors or incomplete conditional logic exist.	Fails to prompt for inputs or store them correctly. Data type conversion is missing/incorrect, leading to runtime errors.
User Interface & Output Formatting	Console output is exceptionally clear and well-organized. Uses formatting utilities (f-strings, \t, \n) expertly to present quantitative results concisely.	Output presents quantitative evidence legibly and correctly formats text and numbers according to target	Output contains raw or unformatted values (e.g., unrounded floats), lacks clear labels, or exhibits minor visual formatting issues.	Fails to provide explicit numerical output, uses vague non-quantitative language, or produces completely incomprehensible output.

Criterion	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations
		specifications.		
Planning Document (yourName_planning)	Planning file is fully complete, highly detailed, and thoroughly models the system using a clear Input-Processing-Output (IPO) framework and accurate pseudocode.	Complete planning document provided containing both an IPO model and pseudocode that accurately reflect the program design.	Planning document is provided but incomplete, missing either the IPO model or pseudocode, or contains significant structural gaps.	Planning document is missing entirely.
Architectural Reflection (yourName_reflection)	Reflection answers all assigned conceptual questions with exceptional depth, clearly explaining rationale for loop selection, data structures, and trade-offs.	Reflection document is provided and concisely answers all assigned questions regarding loop selection and data structure choices.	Reflection document is submitted but incomplete, vague, or fails to address all required conceptual questions.	Reflection document is missing entirely.
Header Comments & Academic Integrity <i>(Zero grade penalty if missing)</i>	Includes a fully formatted header comment at the top of the main code file with all required details and	Header comment is present with required student and project details. AI	Header comment is present but missing key required details. AI chat log was	Header comment is completely missing <i>(Zero grade penalty)</i> . Prohibited external

Criterion	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations
	descriptive context. AI companion chat log (yourName_aichat.txt) is attached if used.	companion chat log is included if applicable.	referenced but not submitted.	AI/human aid used.
File, Folder, & Archive Submission	Folder directory, file extensions, and compressed .zip archive adhere flawlessly to exact naming and structural guidelines.	All files are correctly named, organized inside the specified main project folder, and submitted as a valid .zip archive.	Submission organization or naming is slightly off (e.g., minor typos in filenames, incorrect extension on folder), but files are recognizable .	Submission organization is incorrect, files are loose or misnamed, or required deliverables cannot be located/opened .